

Subscription Autopilot

An AI agent that runs your recurring service spending on Arc — and can prove it behaved.

Ignyte × Circle × Arc · Track 4 (Agentic Economy) · Ronie Joseph ·
roniejosephv@gmail.com · Arc Testnet, chain 5042002, gas = USDC

Circle's official sample shows an agent that *can* pay. Autopilot is an agent that pays **well** — and every dollar it moves passes through an out-of-process policy authority. **The agent can want anything; it can only pay what policy allows.**

WHAT IT DOES

RESEARCH

Probes competing x402 sellers' live price menus (free quotes) against real usage history.

NEGOTIATE

Compares volume/loyalty tiers across sellers at every renewal — switched at 21.9% cheaper per call in the demo.

EXECUTE

Gasless USDC nanopayments per call (x402), batch-settled by Circle Gateway on Arc.

THE CUSTODY BOUNDARY

UNTRUSTED — THE AGENT

LLM, injectable. No keys, no Circle SDK, no chain RPC — only POST /pay with a stated reason. A compromised agent can ask; it cannot *take*.

CUSTODY — SPENDGUARD

allowlist → perTx → monthly → perService
→ velocity → threshold

allow → pay · hold → human · deny → code

Policy versions and spend epochs are auto-anchored on Arc every 10 minutes (SpendAnchor.sol), so every decision is externally verifiable — a tamper-evident audit of the agent's behavior.

CIRCLE STACK — REAL, LOAD-BEARING INTEGRATIONS

Nanopayments / x402	Core rail — per-request dynamic pricing; policy enforced in the onBeforePaymentCreation hook, inside SpendGuard's trusted process.
Circle Wallets (DCW)	Key custody — EIP-712 signing on a developer-controlled EOA; no raw private key exists anywhere in the stack.
Circle Gateway	Settlement + treasury — gas-free EIP-3009 authorizations, batch-settled onchain, transfer UUID receipts.
USDC on Arc	Settlement asset <i>and</i> gas token — audit-anchor transactions are themselves paid in USDC.

VERIFIED ON ARC TESTNET — LIVE DEPLOYMENT

✓ Full E2E loop (Day-0 gate, verify-day0.sh) ✓ DCW signing verified against Gateway settlement ✓ All four demo beats live in production — metering, 21.9% switch, injection denied, human hold/release ✓
SpendAnchor live: 0xfe18f3c42f9318f20cae9cd5b2983e229554e435

autopilotdashboard-production.up.railway.app (live demo) ·
github.com/roniejosephv-star/subscription-autopilot · Apache-2.0 · testnet only

Ignyte × Circle × Arc · Track 4 — Agentic Economy

Subscription Autopilot

An AI agent that runs your recurring service spending on Arc — and can prove it behaved.

Agents can pay — owners can't trust them yet

Capability isn't trust. An LLM with keys in its own process is one bad input away from being someone else's wallet.

prompt_injection

one poisoned instruction and the agent overspends on command

price_drift

sellers reprice constantly — a loyal agent silently wastes money

no_audit_trail

without externally verifiable records, “the agent behaved” is just a claim

Autopilot researches, negotiates, and executes every renewal

01 · RESEARCH

Probes competing x402 sellers' live price menus — free GET /quote — against your real usage history.

02 · NEGOTIATE

Compares volume and loyalty tiers across sellers at every renewal — and switches when cheaper.

−21.9% per call, last switch

03 · EXECUTE

Gasless USDC nanopayments per call (x402), batch-settled onchain by Circle Gateway on Arc.

The agent holds zero keys; policy holds the wallet

UNTRUSTED — THE AGENT

LLM, injectable. No keys, no Circle SDK, no chain RPC. It can only ask: POST /pay with a stated reason.

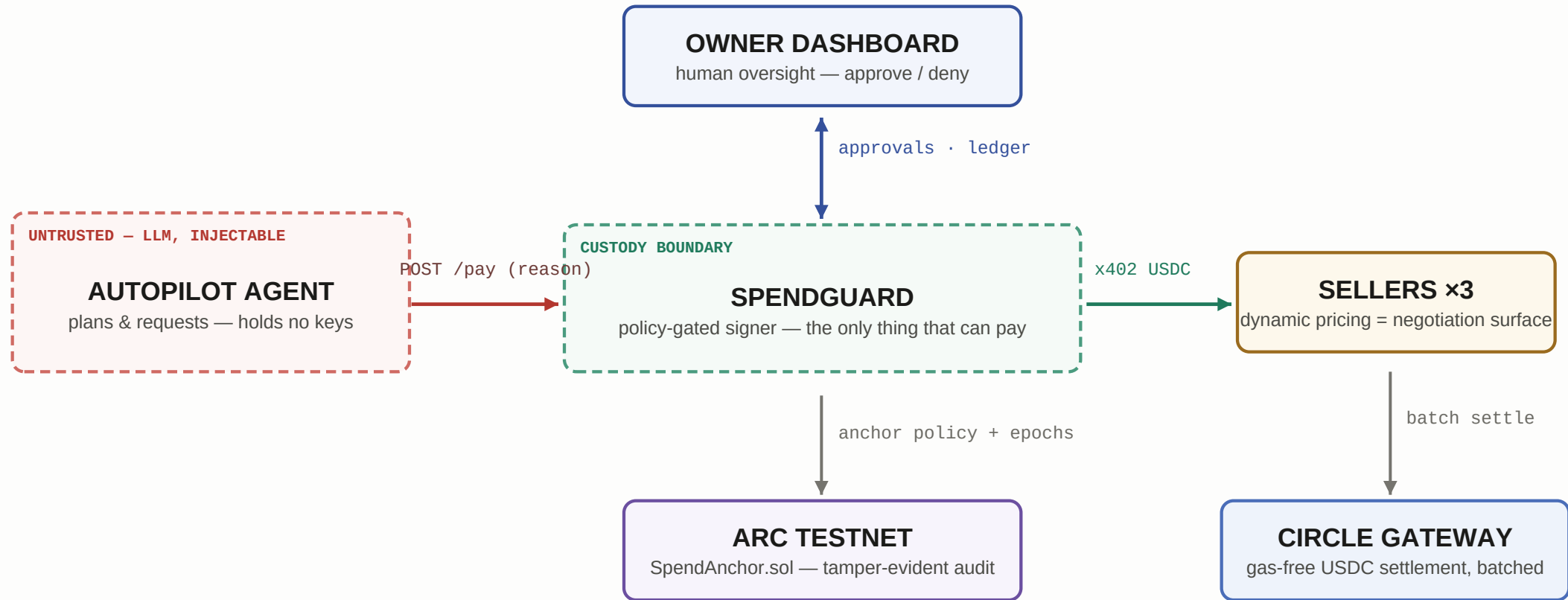
CUSTODY — SPENDGUARD

allowlist → perTx → monthly →
perService → velocity → threshold

allow → pay hold → human deny → code

“The agent can want anything; it can only pay what policy allows.”

Every payment crosses one guarded path



Four Circle products, each doing real work

Nanopayments / x402

Core rail. Seller middleware re-resolves price per request; the `onBeforePaymentCreation` hook runs inside SpendGuard's trusted process — enforcement behind the custody wall.

Circle Wallets (DCW)

Key custody. EIP-712 signing via `signTypedData` on a developer-controlled EOA — no raw private key exists anywhere in the stack.

Circle Gateway

Settlement + treasury. One deposit funds the agent; every payment after is a gas-free EIP-3009 authorization, batch-settled onchain with transfer UUID receipts.

USDC on Arc

Settlement asset *and* gas token — the SpendAnchor audit transactions themselves are paid in USDC on Arc Testnet.

The demo lands in four beats

1 Delegation

agent subscribes to the cheapest seller, meters gasless per-call payments

2 Negotiation

seller raises prices 60% → next renewal re-shops and switches, 21.9% cheaper

3 Attack

prompt-injected overspend → denied at the policy wall (per_tx_max_exceeded), human clicks Deny

4 Proof

month-end burn-down, savings from switching, denial log, receipts anchored on Arc

Verified end-to-end on Arc Testnet

- ✓ Full E2E loop cleared Day 0 — `verify-day0.sh`
- ✓ Circle Wallets DCW signing verified against Gateway settlement — no raw key in the stack
- ✓ All four demo beats live in production: metering, 21.9% switch, injection denied, human hold/release
- ✓ SpendAnchor live — policy hashes + spend epochs auto-anchored every 10 minutes

SPENDANCHOR.SOL · ARC TESTNET

0xfe18f3c42f9318f20cae9cd5b2983e229554e435

Circle's sample shows an agent that can pay.

Autopilot pays well — cheaper, safer, gasless.

Submission details

Track	4 — Best Agentic Economy Experience on Arc
Participant	Ronie Joseph (solo)
Circle Developer Account	roniejosephv@gmail.com
Circle products used	USDC · Wallets (developer-controlled) · Gateway · Nanopayments (x402)
Live demo (dashboard)	https://autopilotdashboard-production.up.railway.app
SpendGuard API (public)	https://autopilotsigner-production.up.railway.app/summary
GitHub repository	https://github.com/roniejosephv-star/subscription-autopilot
Demo video	<i>TODO — paste hosted demo-video URL, then regenerate</i>
On-chain audit anchor	SpendAnchor 0xfe18...e435 on Arc Testnet (arcscan)
Scope	Testnet demo — Arc Testnet, chain 5042002 (gas = USDC)

Working MVP — deployed and verifiable now

Frontend + backend live: the owner dashboard at <https://autopilotdashboard-production.up.railway.app> renders the production ledger (100+ settled nanopayments), budget burn-down, approval queue, policy chain, and the Gateway treasury balance; [the public signer API](#) serves the same data raw. Four services on Railway (SpendGuard signer, 3 x402 sellers, agent, dashboard), SQLite ledger on a persistent volume. Custody mode is SIGNER_MODE=circle — EIP-712 via Circle Wallets DCW EOA, verified against Gateway settlement on Arc Testnet (transfer 9bd6a149-1821-4341-a4aa-712cdc362382, 2026-07-06). Spend epochs auto-anchor on Arc every 10 minutes.

Demo guide (the video's four beats, reproducible live)

1 · Delegation	Agent researches quotes, subscribes to the cheapest seller, meters per-call gasless nanopayments.
2 · Negotiation	Seller price drift → at renewal the agent re-quotes all sellers at real volume and switches (21.9% cheaper per call).
3 · Attack	Prompt-injected overspend → denied at the per-tx policy wall; a \$0.03 premium call is held → human Approve/Deny on the dashboard.
4 · Proof	Burn-down, savings, denial log, Gateway receipts — and the epoch chip linking to the SpendAnchor tx on arcscan.

Documentation

README.md — zero-to-running setup and per-product integration notes with file references · ARCHITECTURE.md — component spec and trust boundaries · PRD.md — product spec, acceptance criteria, risk log · DEPLOYMENT.md — the exact deployment runbook used · docs/demo-script.md — video script · docs/submission/ — this document's sources.

Judge quick-verify (no setup): open the dashboard; curl <https://autopilotsigner-production.up.railway.app/summary>; click the SpendAnchor link on arcscan.

Circle Product Feedback

Why we chose these products

Nanopayments/x402 is the only rail where per-call metered agent payments are economically sane (gasless, sub-cent, batched). Gateway gives the agent a fundable balance with instant receipts. Circle Wallets (DCW EOA) moves key custody out of the agent entirely — the property our whole trust model rests on. USDC-as-gas on Arc collapses faucet/gas UX to one token.

What worked well

- DCW EOA signTypedData signatures are fully ecrecover-compatible with Gateway's x402 settle — the custody-separation design works E2E on Arc Testnet (verified 2026-07-06, transfer 9bd6a149-...-712cdc362382).
- Seller monetization is genuinely 2 lines (createGatewayMiddleware + gateway.require()); per-request dynamic pricing works by binding require(price) at call time — it became our negotiation surface.
- CHAIN_CONFIGS exports eliminated all address/RPC hardcoding.
- Arc deposit finality (~0.5 s) makes the deposit→pay loop feel instant; Wallets contract executions confirm in well under a minute.
- The x402 lifecycle hooks are an excellent extension surface — our entire policy layer lives in one hook.

What could be improved (all found and reproduced during this build)

1. **Nanopayments is EOA-only** (ecrecover on EIP-3009; EIP-1271/SCA unsupported) — easy to miss: Circle's own arc-escrow sample creates accountType SCA wallets, which would silently fail here. Needs a prominent docs callout + runtime hint.
2. **Signature-validity contradiction:** seller quickstart says ≥ 7 days, the SDK error reference says ≥ 3 , and the exported GATEWAY_AUTH_VALIDITY_WINDOW_SECONDS constant is undocumented. Pick one number, document the constant.
3. **18-decimal gas vs 6-decimal ERC-20 USDC on Arc** deserves an explicit docs callout — it bites any native-balance math.
4. **Sample apps are heavy references:** arc-escrow needs Supabase + Docker + ngrok + OpenAI before first run; a minimal single-process reference per product would cut onboarding dramatically.
5. **Undocumented header in shipped types:** X-ARC-PRIVATE-MAINNET-ENABLED appears in the SDK's .d.ts, nowhere in docs.
6. **Hook documentation lives off-site** (x402.org, not developers.circle.com) — mirror or link prominently; it's the SDK's best surface.
7. **Broken script in arc-fintech sample:** package.json declares spend:gateway:arc → a file that doesn't exist in the repo.
8. **Wallets ↔ x402 SDK interop wart:** signTypedData requires an explicit EIP712Domain entry in types, but viem-style callers (including BatchEvmScheme's signer interface in Circle's own SDK) omit it. The error — “there is extra data provided in the message (0 < 4)” — names neither the missing type nor the fix. Accept viem-normalized typed data, or return “types.EIP712Domain is required”.
9. **GatewayClient.deposit() requires a raw private key**, so DCW-custodied wallets can't use it. Verified workaround: replicate its two calls (USDC.approve + GatewayWallet.deposit) via createContractExecutionTransaction. A documented DCW-native deposit path is the single change that would most help agentic use cases — Circle-custodied buyers are exactly who agents need.

Recommendations

Ship a first-class “agent buyer” recipe: DCW EOA wallet + Gateway deposit via contract execution + x402 client with a policy hook — the five pieces exist today but must be discovered across four doc sites and two SDKs' type definitions. This project is, in effect, that recipe; we'd be glad if it were useful as a reference.
